

Human review packet: GS62CollegeAdmissions

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GS62CollegeAdmissions

Paper id: GS62CollegeAdmissions

Source version: American Mathematical Monthly 69(1), January 1962, printed pages 9-15

Source record: audit/paper_statement_map.json

Rows: 5

Generated: 2026-08-18

Source URL: [official source](#)

Review status: 0/5 source-claim screens; 0/6 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GS62CollegeAdmissions --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- [Semantic library prerequisites \(6; not paper claims\)](#)
 - [EconCSLib.FiniteChoice.linearTopQChoice](#)
 - [Assignment M W](#)
 - [ManyToOne.valApplicant](#)
 - [ManyToOneAssignment Applicants Colleges](#)
 - [valM](#)
 - [valW](#)
- [Main-text source claims \(5\)](#)
 - [unstableCollegeAssignment_iff_source_definitionSpec](#)
 - [literalApplicantOptimalCollegeAssignment_iff_source_definitionSpec](#)
 - [unstableMarriage_iff_source_definitionSpec](#)
 - [theorem1_stable_marriage_existsSpec](#)
 - [theorem2_applicant_optimalitySpec](#)

Material library prerequisites

EconCSLib.FiniteChoice.linearTopQChoice

Source locator: cited publication, lines 232-249

Verbatim paper-source connection

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

forward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the

[Next verbatim source excerpt]

return to the philosophical principle mentioned earlier and give it a precise formulation.

DEFINITION. A stable assignment is called optimal if every applicant is at least as well off under it as under any other stable assignment.

[Next verbatim source excerpt]

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

cedures will be the same only when there is a unique stable set of marriages.

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straight-

Lean-expanded library semantic target

```
{α : Type u_1} →  
[inst : DecidableEq α] →  
[inst_1 : LinearOrder α] →  
(q : ℕ) →  
(a : Finset α) →  
if h : q ≤ a.card then Finset.image (fun i => ↑((a.orderIsoOffFin ⋯) (↑i, ⋯))) Finset.univ else a
```

Exact Lean library declaration

```
noncomputable def linearTopQChoice (q : ℕ) : ChoiceRule α :=  
fun X =>  
if h : q ≤ X.card then  
Finset.univ.image  
(fun i : Fin q =>  
((X.orderIsoOffFin (by rfl)  
(i.1, lt_of_lt_of_le i.2 h) : X) : α))  
else  
X
```

Recorded source-to-library screening **Verdict:** matches

Reason: The source says each college retains the q applicants it ranks highest, or all applicants if fewer apply. The Lean library function is exactly that top- q selection from the candidate pool under the college order.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Assignment M W

Source locator: cited publication, lines 133-139

Verbatim paper-source connection

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

Lean declaration type (metadata)

Type u_1 → Type u_2 → Type (max u_1 u_2)

Exact Lean library declaration

```
/-- A matching between Men and Women. -/
structure Assignment (M W : Type*) where
  m_match : M → Option W
  w_match : W → Option M
  consistent_m : ∀ m w, m_match m = some w ↔ w_match w = some m
```

Recorded source-to-library screening Verdict: matches

Reason: The source's set of marriages pairs each participant with an actual mate. The Lean structure records both partner maps and their mutual consistency; the PaperInterface predicate separately states the source's complete-marriage domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ManyToOne.valApplicant

Source locator: cited publication, lines 232-249

Verbatim paper-source connection

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

forward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the

[Next verbatim source excerpt]

return to the philosophical principle mentioned earlier and give it a precise formulation.

DEFINITION. A stable assignment is called optimal if every applicant is at least as well off under it as under any other stable assignment.

[Next verbatim source excerpt]

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

cedures will be the same only when there is a unique stable set of marriages.

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straight-

Lean-expanded library semantic target

```
{Applicants : Type u_1} →
{Colleges : Type u_2} →
(val_applicant : Applicants → Colleges → ℝ) →
(a : Applicants) →
(c : Option Colleges) →
  match c with
  | none => 0
  | some c' => val_applicant a c'
```

Exact Lean library declaration

```
def valApplicant {Applicants Colleges : Type*}
  (val_applicant : Applicants → Colleges → ℝ)
  (a : Applicants) (c : Option Colleges) : ℝ :=
  match c with
  | none => 0
  | some c' => val_applicant a c'
```

Recorded source-to-library screening Verdict: matches

Reason: Sections 4–5 make a terminal applicant either waitlisted or rejected by every willing and permitted college. The procedure's applicant comparison uses an explicit zero outside option for that unmatched terminal case; when matched, the definition returns exactly the applicant's value for the assigned college.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

ManyToOneAssignment Applicants Colleges

Source locator: cited publication, lines 93-98

Verbatim paper-source connection

return to the philosophical principle mentioned earlier and give it a precise formulation.

DEFINITION. A stable assignment is called optimal if every applicant is at least as well off under it as under any other stable assignment.

Lean declaration type (metadata)

Type u_1 → Type u_2 → Type (max u_1 u_2)

Exact Lean library declaration

```
-- A many-to-one assignment between applicants and colleges. -/  
structure ManyToOneAssignment (Applicants Colleges : Type*) where  
  app_match : Applicants → Option Colleges  
  college_roster : Colleges → Finset Applicants  
  consistent : ∀ a c, app_match a = some c ↔ a ∈ college_roster c
```

Recorded source-to-library screening Verdict: matches

Reason: The source describes applicants assigned to colleges and later permits an applicant to be rejected by every college willing to consider them. The Lean structure records the applicant-to-college relation, college rosters, and their consistency, including that permitted unmatched outcome.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

valM

Source locator: cited publication, lines 133-139

Verbatim paper-source connection

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

Lean-expanded library semantic target

```
{M : Type u_1} →
{W : Type u_2} →
(val : M → W → ℝ) →
(m : M) →
(w : Option W) →
  match w with
  | none => 0
  | some w' => val m w'
```

Exact Lean library declaration

```
-- The value of a match for a man. 0 if unmatched. -/
def valM {M W : Type*} (val : M → W → ℝ) (m : M) (w : Option W) : ℝ :=
  match w with
  | none => 0
  | some w' => val m w'
```

Recorded source-to-library screening Verdict: matches

Reason: On the explicitly complete marriage domain of the reviewed source claim, the only relevant branch returns the source man's value for his actual mate; the none-to-zero branch does not represent a source actual mate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

valW

Source locator: cited publication, lines 133-139

Verbatim paper-source connection

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

Lean-expanded library semantic target

```
{M : Type u_1} →
{W : Type u_2} →
(val : W → M → ℝ) →
(w : W) →
(m : Option M) →
  match m with
  | none => 0
  | some m' => val w m'
```

Exact Lean library declaration

```
-- The value of a match for a woman. 0 if unmatched. -/
def valW {M W : Type*} (val : W → M → ℝ) (w : W) (m : Option M) : ℝ :=
  match m with
  | none => 0
  | some m' => val w m'
```

Recorded source-to-library screening Verdict: matches

Reason: On the explicitly complete marriage domain of the reviewed source claim, the only relevant branch returns the source woman's value for her actual mate; the none-to-zero branch does not represent a source actual mate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 1

Source locator: cited publication, lines 93-98

Verbatim source input

return to the philosophical principle mentioned earlier and give it a precise formulation.

DEFINITION. A stable assignment is called optimal if every applicant is at least as well off under it as under any other stable assignment.

Expanded PaperInterface specification

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
  (val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  GS62CollegeAdmissions.PaperInterface.unstableCollegeAssignment val_applicant val_college mu ↔
    ∃ alpha beta A B,
      mu.app_match alpha = some A ∧
      mu.app_match beta = some B ∧
      val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.unstableCollegeAssignment_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec states the displayed current-assignment replacement pair directly: two currently assigned applicants, their current colleges, the applicant's strict improvement, and the college's strict preference for the replacement applicant. It adds no vacancy, unmatched-applicant, or individual-rationality condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 115-117

Verbatim source input

3. Stable assignments and a marriage problem. In trying to settle the question of the existence of stable assignments we were led to look first at a special case, in which there are the same number of applicants as colleges and all

Expanded PaperInterface specification

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} (val_applicant : Applicants → Colleges → ℝ)
(val_college : Colleges → Applicants → ℝ) (mu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
GS62CollegeAdmissions.PaperInterface.literalApplicantOptimalCollegeAssignment val_applicant val_college mu ↔
(¬∃ alpha beta A B,
  mu.app_match alpha = some A ∧
  mu.app_match beta = some B ∧
  val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta) ∧
∀ (nu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
(¬∃ alpha beta A B,
  nu.app_match alpha = some A ∧
  nu.app_match beta = some B ∧
  val_applicant beta B < val_applicant beta A ∧ val_college A alpha < val_college A beta) →
∀ (a : Applicants),
(match nu.app_match a with
| none => 0
| some c => val_applicant a c) ≤
match mu.app_match a with
| none => 0
| some c => val_applicant a c
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.literalApplicantOptimalCollegeAssignment_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec preserves the literal page-10 comparison class: quota-feasible assignments with no displayed replacement pair and applicant-by-applicant weak preference for the selected assignment. This row intentionally does not use the Section 4 operational stability convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 133-139

Verbatim source input

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

Expanded PaperInterface specification

```
∀ {M : Type u_1} {W : Type u_2} (val_m : M → W → ℝ) (val_w : W → M → ℝ) (mu : EconCSLib.Matching.Assignment M W),
  GS62CollegeAdmissions.PaperInterface.unstableMarriage val_m val_w mu ↔
  ((∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m) ∧
  ∃ m w,
    EconCSLib.Matching.valM val_m m (mu.m_match m) < val_m m w ∧
    EconCSLib.Matching.valW val_w w (mu.w_match w) < val_w w m
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.unstableMarriage_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The source's marriage discussion concerns a complete matching and a man-woman pair who each prefer the other to the present partner. The transparent Spec writes the completeness and both strict present-partner comparisons explicitly.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 127-139

Verbatim source input

mates.

QUESTION: For any pattern of preferences is it possible to find a stable set of marriages?

Before giving the answer let us look at some examples.

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

[Next verbatim source excerpt]

,y ranks a first, and a, 3 and y all rank a last. Then regardless of S's preference

[Next verbatim source excerpt]

mates.

QUESTION: For any pattern of preferences is it possible to find a stable set of marriages?

Before giving the answer let us look at some examples.

[Next verbatim source excerpt]

Example 1. The following is the "ranking matrix" of three men, a, ,3, and 7, and three women, A, B, and C.

	A	B	C
a	1,3	2,2	3,1

Expanded PaperInterface specification

```
∀ {M : Type u_1} {W : Type u_2} [inst : Fintype M] [inst_1 : Fintype W] [DecidableEq M] [DecidableEq W]
  (val_m : M → W → ℝ) (val_w : W → M → ℝ),
  Fintype.card M = Fintype.card W →
  (∀ (m : M) (w w' : W), val_m m w = val_m m w' → w = w') →
  (∀ (w : W) (m m' : M), val_w w m = val_w w m' → m = m') →
  (∀ (m : M) (w : W), 0 < val_m m w) →
  (∀ (w : W) (m : M), 0 < val_w w m) →
  ∃ mu,
  ¬((∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m) ∧
  ∃ m w,
  EconCSLib.Matching.valM val_m m (mu.m_match m) < val_m m w ∧
  EconCSLib.Matching.valW val_w w (mu.w_match w) < val_w w m) ∧
  (∀ (m : M), ∃ w, mu.m_match m = some w) ∧ ∀ (w : W), ∃ m, mu.w_match w = some m
```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.theorem1_stable_marriage_exists

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent Spec has the source's finite equal-size strict complete domain and concludes a complete assignment with no mutually preferred blocking pair. Its expanded form makes the source's stable-marriage claim checkable without a wrapper predicate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 232-249

Verbatim source input

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

forward. For convenience we will assume that if a college is not willing to accept a student under any circumstances, as described in Section 2, then that student will not even be permitted to apply to the college. With this understanding the

[Next verbatim source excerpt]

return to the philosophical principle mentioned earlier and give it a precise formulation.

DEFINITION. A stable assignment is called optimal if every applicant is at least as well off under it as under any other stable assignment.

[Next verbatim source excerpt]

a 1, 3 2, 2 3, 1 4, 3

3 1,4 2,3 3,2 4,4

y 3,1 1,4 2,3 4,2
5 2,2 3,1 1,4 4,1

The condition that there be the same number of boys and girls is not essential. If there are b boys and g girls with $b < g$, then the procedure terminates as soon as b girls have been proposed to. If $b > g$ the procedure ends when every boy is either on some girl's string or has been rejected by all of the girls. In either case the set of marriages that results is stable.

It is clear that there is an entirely symmetrical procedure, with girls proposing to boys, which must also lead to a stable set of marriages. The two solutions are not generally the same as shown by Example 1; indeed, we shall see in a moment that when the boys propose, the result is optimal for the boys, and when the girls propose it is optimal for the girls. The solutions by the two pro-

[Next verbatim source excerpt]

cedures will be the same only when there is a unique stable set of marriages.

4. Stable assignments and the admissions problem. The extension of our "deferred-acceptance" procedure to the problem of college admissions is straight-

Expanded PaperInterface specification

```
∀ {Applicants : Type u_1} {Colleges : Type u_2} [inst : Fintype Applicants] [inst_1 : Fintype Colleges]
[inst_2 : DecidableEq Applicants] [inst_3 : DecidableEq Colleges] (quota : Colleges → ℕ)
(val_applicant : Applicants → Colleges → ℝ) (val_college : Colleges → Applicants → ℝ),
((∀ (a : Applicants) (c c' : Colleges), val_applicant a c = val_applicant a c' → c = c') ∧
  ∀ (a : Applicants) (c : Colleges), val_applicant a c ≠ 0) →
  ∀
    (hcollege_strict :
      (∀ (c : Colleges) (a a' : Applicants), val_college c a = val_college c a' → a = a') ∧
      ∀ (c : Colleges) (a : Applicants), val_college c a ≠ 0),
    (∃ s,
      GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable quota val_applicant val_college = s ∧
      ¬ ∃ a,
        GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive quota val_applicant val_college = s a) ∧
    ∀ (s : GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceWaitingListState Applicants Colleges),
      GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceReachable quota val_applicant val_college = s →
      (¬ ∃ a,
        GS62CollegeAdmissions.ExactCollegeBatchedProcedure.SourceActive quota val_applicant val_college = s
          a) →
      ((∀ (c : Colleges),
        ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college =
          s).college_roster
            c).card ≤
          quota c) ∧
        (∀ (a : Applicants),
          0 ≤
```

```

EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
  ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
    s).app_match
    a)) ∧
(∀ (c : Colleges),
  ∀
    a ∈
      (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
        s).college_roster
        c,
    0 ≤ val_college c a) ∧
∀ (a : Applicants) (c : Colleges),
  EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
    ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
      val_college ... s).app_match
      a) <
    val_applicant a c →
    (0 < val_college c a ∧
      ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
        val_college ... s).college_roster
        c).card <
        quota c v
      ∃
        a' ∈
          (GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota
            val_college ... s).college_roster
            c,
          val_college c a' < val_college c a) →
      False) ∧
∀ (nu : EconCSLib.Matching.ManyToOneAssignment Applicants Colleges),
  ((∀ (c : Colleges), (nu.college_roster c).card ≤ quota c) ∧
    (∀ (a : Applicants),
      0 ≤ EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a)) ∧
    (∀ (c : Colleges), ∀ a ∈ nu.college_roster c, 0 ≤ val_college c a) ∧
    (∀ (a : Applicants) (c : Colleges),
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a) <
        val_applicant a c →
        (0 < val_college c a ∧ (nu.college_roster c).card < quota c v
          ∃ a' ∈ nu.college_roster c, val_college c a' < val_college c a) →
        False) →
    (∀ (a : Applicants),
      EconCSLib.Matching.ManyToOne.valApplicant val_applicant a (nu.app_match a) ≤
        EconCSLib.Matching.ManyToOne.valApplicant val_applicant a
          ((GS62CollegeAdmissions.ExactCollegeBatchedProcedure.sourceAssignmentView quota val_college ...
            s).app_match
            a)
    ))

```

Lean proof endpoint (built separately)

GS62CollegeAdmissions.ProofInterface.theorem2_applicant_optimality

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle includes both the earlier displayed replacement-pair condition and Sections 4–5. Read together, the latter uses stable assignment for the terminal top-quota waiting-list outcome and says its stability proof is analogous to the marriage proof; it therefore supplies the completed operational comparison class used by the transparent Spec. The literal display remains a distinct source claim and is not silently identified with that Section 4–5 reading.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation